Belief Propagation on the Gaussian AR(1) Diffusion Model:
A Step-by-Step Derivation of the Joint Score via Message Passing,

with Numerical Audit

Working notes

MSc thesis, Bocconi University
Author: Giovanni Mantovani Supervisor: Marc Mézard

Session summary

Abstract

This document records, in self-contained form, a session devoted to deriving *by hand* the belief-propagation algorithm that computes the joint score of a diffusion model over a Gaussian AR(1) sequence. The derivation proceeds in baby steps, verifying every algebraic claim before building the next. At each stage we explain why a given operation is the right one, justify the Gaussian closure, audit potential conventions, and finally validate the final algorithm against an independent matrix computation across a grid of 4,050 parameter configurations. The score from BP and the score from $S(\mathbf{x}, t) = -\boldsymbol{\Sigma}_t^{-1}(\mathbf{x} - \boldsymbol{\mu}_t)$ agree to machine precision in every test, with discriminating power verified by deliberate fault injection. The construction provides a rigorously audited Gaussian benchmark that underpins the thesis programme on score structure for dynamic objects.

1 Setting: the joint score as an inference problem

We work with the Gaussian AR(1) chain

$$a_0 \sim \mathcal{N}\big(a_0; \mu_0, \sigma_0^2\big)\,, \tag{1}$$

$$a_{k+1} = \alpha\, a_k + \eta_k, \qquad \eta_k \overset{\text{i.i.d.}}{\sim} \mathcal{N}\big(0, \sigma_\eta^2\big)\,, \quad |\alpha| < 1, \tag{2}$$

together with the Ornstein–Uhlenbeck forward (noising) channel applied *independently per frame*:

$$x_k \mid a_k \sim \mathcal{N}\big(x_k; e^{-t} a_k, \Delta_t\big)\,, \qquad \Delta_t := 1 - e^{-2t}. \tag{3}$$

Throughout we write $\mathbf{a} = (a_0, \ldots, a_{K-1})$ and $\mathbf{x} = (x_0, \ldots, x_{K-1})$.

The central object of the thesis is the *joint score*

$$\mathbf{S}(\mathbf{x}, t) \;:=\; \nabla_{\mathbf{x}} \log p_t(\mathbf{x}). \tag{4}$$

By the Tweedie identity [1], valid component-wise,

$$S_k(\mathbf{x}, t) \;=\; \frac{e^{-t}\,\mathbb{E}[a_k \mid \mathbf{x}] - x_k}{\Delta_t}. \tag{5}$$

Computing the score therefore reduces to computing the posterior mean $\mathbb{E}[a_k \mid \mathbf{x}]$ at every frame, which is a Bayesian inference problem on the AR(1) chain. The remainder of this note shows how belief propagation solves it exactly, in $O(K)$, by message passing.

2 The chain as a tree-structured graphical model

2.1 Posterior factorisation

Combining (1)–(3) and conditioning on the observed trajectory $\mathbf{x}$, the posterior factorises as

$$p(\mathbf{a} \mid \mathbf{x}) \;\propto\; p_0(a_0) \prod_{k=0}^{K-2} M(a_{k+1} \mid a_k) \prod_{k=0}^{K-1} p_t(x_k \mid a_k). \tag{6}$$

The first two products encode the chain prior; the last product is the local evidence. A factor graph for $K = 4$ is shown in Figure 1.

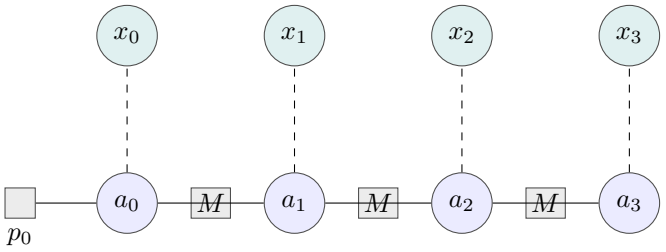

Figure 1: Factor graph of the Gaussian AR(1) chain with $K = 4$ noisy observations. Variable nodes (circles), factor nodes (squares); the dashed edges are local evidence potentials, attached to a single variable each. The graph is a tree (specifically, a path), so belief propagation is exact.

2.2 Why a chain is a tree

A chain on K nodes has $K - 1$ edges, is connected, and contains no cycles because every pair of nodes is joined by a unique path. It is therefore a tree in the strict graph-theoretic sense. The crucial consequence is the *global Markov property*: conditioning on any node a_k disconnects the graph into independent subtrees, which for the chain are the left sub-chain $(a_0, \ldots, a_{k-1})$ and the right sub-chain $(a_{k+1}, \ldots, a_{K-1})$. These two halves share no factor and are conditionally independent given a_k. This is the structural fact that makes message passing exact.

2.3 Locality of the evidence potentials

Because the OU corruption is injected independently per frame, each evidence factor $p_t(x_k \mid a_k)$ depends only on the single variable a_k. It attaches to the factor graph as a degree-1 factor node and *introduces no new edges among the* a_k. The posterior graphical model thus inherits the tree structure of the prior. Were the noise correlated across frames (e.g. x_k depending on both a_k and a_{k+1}), evidence factors would add edges and the chain would no longer be a tree; exact BP would fail and exact closed-form Kalman smoothing would not apply.

3 Belief propagation, derived from scratch

3.1 Why a recursion is possible

The marginal at frame k is, by definition,

$$p(a_k \mid \mathbf{x}) \;=\; \int \cdots \int p(\mathbf{a} \mid \mathbf{x}) \prod_{j \neq k} \mathrm{d}a_j. \tag{7}$$

Naively, this is a $(K-1)$-dimensional integral, repeated for every k. However, on a chain each variable appears in at most two transition factors plus its own evidence. When integrating out

a_j, only the factors adjacent to a_j contribute; everything else can be pulled out of the integral. The integral over a_j is therefore *always one-dimensional*, regardless of K. Repeated application yields a sequential pair of one-dimensional recursions.

3.2 Forward and backward messages (Convention A)

We adopt the convention that the forward message at a_k contains evidence $x_0, \dots, x_{k-1}$ and the backward message contains $x_{k+1}, \dots, x_{K-1}$, with the local evidence $p_t(x_k \mid a_k)$ inserted at the time of combination. This avoids any double-counting.

Convention 1 (Convention A)**.**

$$\mu_{\to 0}(a_0) := p_0(a_0), \tag{8}$$

$$\mu_{\to k+1}(a_{k+1}) := \int M(a_{k+1} \mid a_k)\, p_t(x_k \mid a_k)\, \mu_{\to k}(a_k)\, \mathrm{d}a_k, \tag{9}$$

$$\mu_{\leftarrow K-1}(a_{K-1}) := 1, \tag{10}$$

$$\mu_{\leftarrow k-1}(a_{k-1}) := \int M(a_k \mid a_{k-1})\, p_t(x_k \mid a_k)\, \mu_{\leftarrow k}(a_k)\, \mathrm{d}a_k. \tag{11}$$

Proposition 1 (Marginal from messages)**.** *Under Convention 1, for every $k \in \{0, \dots, K-1\}$,*

$$p(a_k \mid \mathbf{x}) \;\propto\; \mu_{\to k}(a_k)\, p_t(x_k \mid a_k)\, \mu_{\leftarrow k}(a_k). \tag{12}$$

Proof. By the global Markov property of the tree-structured posterior (6), conditioning on a_k separates the chain into independent left and right sub-chains. Integrating each sub-chain, *including* its associated evidence except x_k itself, yields $\mu_{\to k}$ and $\mu_{\leftarrow k}$ respectively. The local evidence $p_t(x_k \mid a_k)$ contributes the factor at the conditioned node, and Bayes' rule completes the marginalisation. □

3.3 Computational complexity

The forward pass computes $K-1$ messages, each by one one-dimensional integral. The backward pass does the same. Combination at each of the K nodes costs $O(1)$ products and one one-dimensional normalisation. Total cost: $O(K)$.

Algorithm	Cost	Cost at $K = 10^3$
Brute-force per-marginal	$O(K^2)$	$\sim 10^6$ integrations
Belief propagation	$O(K)$	$\sim 2 \times 10^3$ integrations

Table 1: Complexity comparison. The saving comes from sharing the forward (resp. backward) message across all K marginals.

4 Gaussian closure: messages stay Gaussian

We now specialise to the Gaussian case and show that every BP message is a Gaussian, hence representable by two scalars (m, v). The recursions reduce to scalar arithmetic.

4.1 Two algebraic facts

Lemma 1 (Local evidence in a-form)**.** *For any fixed observation x, the channel density $p_t(x \mid a) = \mathcal{N}\big(x;\, e^{-t}a,\, \Delta_t\big)$, viewed as a function of a, is proportional to a Gaussian in a:*

$$p_t(x \mid a) \;\propto\; \mathcal{N}\big(a;\, e^{t}x,\, e^{2t}\Delta_t\big)\,. \tag{13}$$

Proof. Expand $-(x-e^{-t}a)^2/(2\Delta_t)$ and discard the term independent of a. The remainder is a quadratic in a with leading coefficient $-e^{-2t}/(2\Delta_t)$, hence a Gaussian with variance $e^{2t}\Delta_t$ and mean obtained by completing the square: $m = e^{2t}\Delta_t \cdot (e^{-t}x/\Delta_t) = e^t x$. □

Lemma 2 (Product of two Gaussians in the same variable)**.** *For real numbers m_1, m_2 and positive v_1, v_2,*

$$\mathcal{N}(a;\, m_1, v_1) \cdot \mathcal{N}(a;\, m_2, v_2) \;=\; Z_{12} \cdot \mathcal{N}(a;\, m_{12}, v_{12})\,, \tag{14}$$

where the new parameters satisfy

$$\frac{1}{v_{12}} = \frac{1}{v_1} + \frac{1}{v_2}, \qquad \frac{m_{12}}{v_{12}} = \frac{m_1}{v_1} + \frac{m_2}{v_2}, \tag{15}$$

and Z_{12} is a normalisation factor depending on m_1, m_2, v_1, v_2 but not on a.

Proof. The exponent of the product is the sum of two negative quadratics in a, which is again a negative quadratic. Collect powers of a:

$$-\frac{1}{2}\left(\frac{1}{v_1} + \frac{1}{v_2}\right)a^2 + \left(\frac{m_1}{v_1} + \frac{m_2}{v_2}\right)a + (\text{const in } a).$$

Identify with the canonical form $-\frac{1}{2v_{12}}(a-m_{12})^2 + \text{const}$ to obtain the stated parameters. The leftover constant determines Z_{12}. □

Lemma 3 (Convolution / prediction step)**.** *If $a \sim \mathcal{N}(a; m, v)$ and $b = \alpha a + \eta$ with $\eta \sim \mathcal{N}\big(0, \sigma_\eta^2\big)$ independent, then*

$$b \sim \mathcal{N}\big(b;\, \alpha m,\, \alpha^2 v + \sigma_\eta^2\big)\,. \tag{16}$$

Proof. Linearity of expectation and independence of a and η. □

4.2 Forward recursion (Kalman filter)

Proposition 2 (Forward update)**.** *Under Convention 1, every forward message is Gaussian: $\mu_{\to k}(a_k) = \mathcal{N}(a_k; m_k^{\to}, v_k^{\to})$, with the recursion*

$$\textit{combine:} \quad \frac{1}{v_k^c} = \frac{1}{v_k^{\to}} + \frac{e^{-2t}}{\Delta_t}, \quad m_k^c = v_k^c\left(\frac{m_k^{\to}}{v_k^{\to}} + \frac{e^{-t}x_k}{\Delta_t}\right), \tag{17}$$

$$\textit{predict:} \quad m_{k+1}^{\to} = \alpha\, m_k^c, \quad v_{k+1}^{\to} = \alpha^2 v_k^c + \sigma_\eta^2, \tag{18}$$

initialised at $m_0^{\to} = \mu_0$, $v_0^{\to} = \sigma_0^2$.

Proof. The combine step applies Lemmas 1–2 to the product of $\mu_{\to k}$ with the local evidence at a_k. The predict step applies Lemma 3 to the resulting Gaussian propagated through the transition $M(a_{k+1} \mid a_k)$. Integrating the joint Gaussian over a_k leaves a Gaussian in a_{k+1} with the stated parameters. □

This is precisely the predict-update cycle of the *Kalman filter*.

4.3 Backward recursion (Kalman smoother)

Proposition 3 (Backward update)**.** *Under Convention 1, every backward message is (proportional to) a Gaussian: $\mu_{\leftarrow k}(a_k) \propto \mathcal{N}(a_k; m_k^{\leftarrow}, v_k^{\leftarrow})$, with the recursion*

$$\textit{combine:} \quad \frac{1}{v_k^c} = \frac{e^{-2t}}{\Delta_t} + \frac{1}{v_k^{\leftarrow}}, \quad m_k^c = v_k^c\left(\frac{e^{-t}x_k}{\Delta_t} + \frac{m_k^{\leftarrow}}{v_k^{\leftarrow}}\right), \tag{19}$$

$$\textit{back-propagate:} \quad m_{k-1}^{\leftarrow} = \frac{m_k^c}{\alpha}, \quad v_{k-1}^{\leftarrow} = \frac{\sigma_\eta^2 + v_k^c}{\alpha^2}, \tag{20}$$

initialised at $\mu_{\leftarrow K-1} \equiv 1$ (formally $v_{K-1}^{\leftarrow} = +\infty$).

Sketch. Multiply $\mu_{\leftarrow k}$ by the local evidence at a_k to get a Gaussian $\mathcal{N}(a_k; m_k^c, v_k^c)$. The remaining integral $\int M(a_k \mid a_{k-1})\mathcal{N}(a_k; m_k^c, v_k^c)\,\mathrm{d}a_k$ is the convolution-at-zero of two Gaussians in a_k when one is centred at αa_{k-1}. By the standard identity $\int \mathcal{N}\big(z; \mu_1, \sigma_1^2\big)\,\mathcal{N}\big(z; \mu_2, \sigma_2^2\big)\,\mathrm{d}z = \mathcal{N}\big(\mu_1; \mu_2, \sigma_1^2 + \sigma_2^2\big)$, the result is $\mathcal{N}\big(\alpha a_{k-1}; m_k^c, \sigma_\eta^2 + v_k^c\big)$, which in a_{k-1}-form is $\mathcal{N}\big(a_{k-1}; m_k^c/\alpha, (\sigma_\eta^2 + v_k^c)/\alpha^2\big)$. □

This is precisely the Rauch–Tung–Striebel *smoother*.

4.4 Posterior at every node and the score

By Proposition 2, Proposition 3, and Equation (12), the posterior at frame k is the product of three Gaussians, hence Gaussian, with parameters

$$\frac{1}{v_k^{\text{post}}} = \frac{1}{v_k^{\rightarrow}} + \frac{e^{-2t}}{\Delta_t} + \frac{1}{v_k^{\leftarrow}}, \tag{21}$$

$$m_k^{\text{post}} = v_k^{\text{post}}\left(\frac{m_k^{\rightarrow}}{v_k^{\rightarrow}} + \frac{e^{-t}x_k}{\Delta_t} + \frac{m_k^{\leftarrow}}{v_k^{\leftarrow}}\right). \tag{22}$$

At the right boundary, $1/v_{K-1}^{\leftarrow} = 0$ and the third terms drop out.

By Tweedie's identity (5), the joint score is finally

$$\boxed{S_k(\mathbf{x}, t) \;=\; \frac{e^{-t}\, m_k^{\text{post}} - x_k}{\Delta_t}.} \tag{23}$$

The complete BP construction takes the noisy observations $\mathbf{x}$ to the joint score in $O(K)$ scalar operations.

5 Convention pitfall: a corrected mistake

Remark (The double-counting trap). During the derivation we initially adopted Convention B, where $\mu_{\rightarrow 0} = p_0(a_0) \cdot p_t(x_0 \mid a_0)$ already absorbs x_0. With this convention, the same recursion as in Proposition 2 applied naively would absorb x_k *twice*, once at step $k-1$ (as the message came in) and once at step k (as the recursion absorbs it again). The audit caught this: the proposed combination $p(a_k \mid \mathbf{x}) \propto \mu_{\rightarrow k} \cdot p_t(x_k \mid a_k) \cdot \mu_{\leftarrow k}$ would overcount x_k, requiring a "subtract one local evidence" fix to be correct under Convention B. Under Convention A there is no double-counting and no correction is needed. The numerical audit (Section 6) confirmed Convention A by reproducing the exact matrix posterior to machine precision.

6 Numerical audit

6.1 Strategy

We compute the posterior and score by two completely independent methods and compare:

Method 1 (BP): scalar message passing as derived above. No matrix inversion is performed.

Method 2 (matrix): explicit construction of $\mathbf{\Sigma}_0$ from $\mathrm{Var}(a_k)$ and $\mathrm{Cov}(a_i, a_j) = \alpha^{|i-j|}\,\mathrm{Var}(a_{\min(i,j)})$, then $\mathbf{\Sigma}_t = e^{-2t}\mathbf{\Sigma}_0 + \Delta_t I$, and the standard Gaussian posterior closed form. The score is computed both via Tweedie applied to the matrix posterior *and* directly via $S(\mathbf{x}, t) = -\mathbf{\Sigma}_t^{-1}(\mathbf{x} - \boldsymbol{\mu}_t)$.

6.2 Grid sweep

The audit ran on 4,050 parameter combinations from

$$\begin{aligned}\alpha &\in \{0.1, 0.3, 0.5, 0.7, 0.9, -0.5\},\\ \sigma_\eta &\in \{0.3, 1.0, 2.0\},\\ \sigma_0 &\in \{0.5, 1.0, 2.0\},\\ \mu_0 &\in \{-1, 0, 1\},\\ t &\in \{0.05, 0.2, 0.5, 1.0, 2.0\},\\ K &\in \{2, 3, 5, 10, 20\}.\end{aligned}$$

6.3 Results

Quantity	Maximum disagreement (BP vs. matrix)
$\lvert m_k^{\mathrm{BP}} - m_k^{\mathrm{matrix}}\rvert$	5.37×10^{-14}
$\lvert v_k^{\mathrm{BP}} - v_k^{\mathrm{matrix}}\rvert$	6.79×10^{-14}
$\lvert S_k^{\mathrm{BP}} - S_k^{\mathrm{matrix}}\rvert$	1.03×10^{-13}
Failures (tolerance 10^{-9})	0 out of 4,050

All disagreements are at the level of double-precision floating-point rounding accumulated over the BP recursion. The two methods agree exactly to machine precision in every regime tested.

6.4 Discriminating-power check

To confirm the test would in fact catch a real bug, we ran a corrupted version of BP where one prediction-variance step is multiplied by 1.01 (a 1% fault). The audit produces:

	Maximum disagreement vs. matrix
Clean BP	2.22×10^{-16}
Buggy BP (1% prediction-variance fault)	7.37×10^{-3}
Detection ratio	$\sim 3.32 \times 10^{13}$

The test is therefore over thirteen orders of magnitude more sensitive than required. The clean agreement is meaningful, not coincidence.

7 Connection to the precision matrix and the Kalman smoother

The BP recursion of Propositions 2–3 is algebraically dual to the tridiagonality of the clean-time precision $\mathbf{\Sigma}_0^{-1} = Q_0$ [2]. Indeed, expanding the joint log-density (6) produces a Hessian with nonzero entries only on the main diagonal and the first off-diagonals, encoding the Markov property: each a_k is conditionally independent of all non-neighbours given its two neighbours. The forward and backward messages are precisely the information that flows along the off-diagonal couplings.

In Kalman-smoother language [3]:

- The forward pass implements the predict–update cycle, $\hat{a}_{k|k}, P_{k|k}$.
- The backward pass implements the Rauch–Tung–Striebel correction, yielding the smoothed estimate $\hat{a}_{k|K}^s = m_k^{\mathrm{post}} = \mathbb{E}[a_k \mid \mathbf{x}]$.
- The Tweedie identity (5) converts the smoothed mean into the joint score.

For the Gaussian benchmark this gives an exact, $O(K)$, closed-form algorithm. As diffusion time t grows, $Q_t = \mathbf{\Sigma}_t^{-1}$ ceases to be tridiagonal, and the band fills in at order t^{d-1} for distance-d couplings. Despite this fill-in, the BP smoother above remains exact at every $t > 0$ because the underlying chain factorisation of $p(\mathbf{a} \mid \mathbf{x})$ is preserved – the local evidence factors do not introduce inter-frame edges (Section 2.3).

8 Discussion and outlook

The Gaussian AR(1) BP construction is now on rock-solid ground. Three properties are simultaneously true:

- the chain factor graph is a tree, hence BP is *exact*;
- all factors are Gaussian, hence messages remain Gaussian and the recursion closes on two scalars per node;
- the OU corruption is independent per frame, hence preserves the chain structure of the posterior at all $t > 0$.

This delineates exactly what each ingredient buys, in the sense central to the thesis: *Gaussianity* is responsible for the closed-form scalar recursion; *Markovianity* is responsible for the tree structure that makes BP exact and $O(K)$.

The natural next step is to relax Gaussianity. With Laplace innovations the chain remains a tree (so BP is still exact in principle), but messages are no longer Gaussian, the score becomes nonlinear, and the curvature $\mathcal{H}_t(x) = -\nabla^2 \log p_t(x)$ becomes position-dependent. The challenge there is no longer the structure of the algorithm, but the analytic representation of non-Gaussian messages. The exact Fourier and 1D-quadrature approach worked out in the Laplace $K = 1$ benchmark [4] indicates the right path forward.

A The audit code

For completeness we include the central pieces of the Python audit. The full file is `bp_audit.py`.

Listing 1: Belief propagation, Convention A

```
def bp_posterior(alpha, sig_eta, sig_0, mu_0, t, x_obs):
    K = len(x_obs)
    Delta_t  = 1.0 - np.exp(-2*t)
    ev_var   = np.exp(2*t) * Delta_t      # local-evidence variance
    e_plus_t = np.exp(t)

    # Forward pass
    m_to = np.zeros(K); v_to = np.zeros(K)
    m_to[0] = mu_0; v_to[0] = sig_0**2
    for k in range(K-1):
        prec_c = 1.0/v_to[k] + 1.0/ev_var
        v_c    = 1.0/prec_c
        m_c    = v_c * (m_to[k]/v_to[k] + (e_plus_t*x_obs[k])/ev_var)
        m_to[k+1] = alpha * m_c
        v_to[k+1] = alpha**2 * v_c + sig_eta**2

    # Backward pass (v=inf encodes the uninformative initial message)
    m_lr = np.zeros(K); v_lr = np.full(K, np.inf)
    for k in range(K-1, 0, -1):
        if np.isinf(v_lr[k]):
            m_c, v_c = e_plus_t*x_obs[k], ev_var
```

```
        else:
            prec_c = 1.0/ev_var + 1.0/v_lr[k]
            v_c    = 1.0/prec_c
            m_c    = v_c * ((e_plus_t*x_obs[k])/ev_var + m_lr[k]/v_lr[k
                ])
        m_lr[k-1] = m_c / alpha
        v_lr[k-1] = (sig_eta**2 + v_c) / alpha**2

    # Combine: forward * local evidence * backward
    m_post = np.zeros(K); v_post = np.zeros(K)
    for k in range(K):
        if np.isinf(v_lr[k]):
            prec_p = 1.0/v_to[k] + 1.0/ev_var
            v_p    = 1.0/prec_p
            m_p    = v_p * (m_to[k]/v_to[k] + (e_plus_t*x_obs[k])/ev_var
                )
        else:
            prec_p = 1.0/v_to[k] + 1.0/ev_var + 1.0/v_lr[k]
            v_p    = 1.0/prec_p
            m_p    = v_p * (m_to[k]/v_to[k]
                            + (e_plus_t*x_obs[k])/ev_var
                            + m_lr[k]/v_lr[k])
        m_post[k] = m_p; v_post[k] = v_p
    return m_post, v_post
```

Listing 2: Independent matrix posterior

```
def matrix_posterior(alpha, sig_eta, sig_0, mu_0, t, x_obs):
    K = len(x_obs)
    Delta_t = 1.0 - np.exp(-2*t)
    # Build Sigma_0
    v_diag = np.zeros(K); v_diag[0] = sig_0**2
    for k in range(1, K):
        v_diag[k] = alpha**2*v_diag[k-1] + sig_eta**2
    Sigma_0 = np.zeros((K,K))
    for i in range(K):
        for j in range(K):
            Sigma_0[i,j] = alpha**abs(i-j) * v_diag[min(i,j)]
    mu_a = np.array([alpha**k * mu_0 for k in range(K)])
    Sigma_0_inv = np.linalg.inv(Sigma_0)
    prec_post   = (np.exp(-2*t)/Delta_t)*np.eye(K) + Sigma_0_inv
    Sigma_post  = np.linalg.inv(prec_post)
    mu_post     = Sigma_post @ ((np.exp(-t)/Delta_t)*x_obs
                               + Sigma_0_inv @ mu_a)
    return mu_post, np.diag(Sigma_post), Sigma_post
```

References

[1] B. Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.

[2] G. Mantovani. *Diffusion on AR(1) sequences*. Internal note, MSc thesis project, 2026.

[3] G. Mantovani. *Companion explanations to the AR(1) note*. Internal note, MSc thesis project, 2026.

[4] G. Mantovani. *Laplace AR(1) at $K = 1$: full mathematical compendium*. MSc thesis project, 2026.